

LEGAL PDF ANALYZER

A Major Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology (Computer Science & Engineering)

Submitted By

Mohammad Raif

Enrollment No.: 2022-310-120

Under the Supervision of

Mr. Nafisur Rahman



Department of Computer Science & Engineering

School of Engineering Sciences & Technology

Jamia Hamdard, New Delhi

Session: 2025-2026



جامعہ ہمدرد جَامِيَا ہَمْدَرْد

JAMIA HAMDARD

(Deemed to be University)
Accredited by NAAC in 'A+' Category

Hamdard Nagar, New Delhi-1100
Phone: 011-26059688 (12 Lines)
Website: www.jamiahamdard.edu

CERTIFICATE

On the basis of the declaration submitted by **Mr. Mohammad Raif** (Enrolment No: 2022-310-120) a student of **B.Tech (Computer Science & Engineering)** I hereby certify that the dissertation titled "**LEGAL PDF ANALYZER**" being submitted to the Department of Computer Science & Engineering, Jamia Hamdard, New Delhi in partial fulfillment of the requirement for the award of the degree of **B.Tech (Computer Science & Engineering)** is carried out by him under my supervision.

Prof. Parul Agarwal
Head, Department of CSE
Dean, SEST.


Mr. Nafisur Rahman
(Supervisor)

Declaration

I, Mohammad Raif, a student of Bachelor of Technology (Computer Science & Engineering) (Enrollment No.:2022-310-120), hereby declare that the dissertation entitled “Legal PDF Analyzer”, submitted to the Department of Computer Science & Engineering, School of Engineering Sciences & Technology, Jamia Hamdard, New Delhi, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology (Computer Science & Engineering), is my original work. This work has not been submitted elsewhere for the award of any degree, diploma, associateship, fellowship, or similar recognition.

Signature and Name of the Applicant: Mohammad Raif

Date: 15.05.26

Place: New Delhi

Acknowledgements

I would like to express my sincere gratitude to my project supervisor and the faculty members of the Department of Computer Science & Engineering for their guidance and encouragement throughout this work. Their feedback helped me shape the project from a simple idea into a usable system with a clear workflow for legal document analysis.

I am also thankful to my classmates and friends who discussed the practical problems faced while reading large legal judgments. Those conversations made the project more grounded. They helped me understand that the goal was not only to process PDFs, but also to make references, relationships, and summaries easier to explore.

Finally, I thank my family for their constant support and patience during the development and documentation of this project. This report is a reflection of both the technical learning and the personal discipline gained during my final year.

Mohammad Raif

Date: _____

List of Tables

Table 3.1 Functional Requirements

Table 3.2 Non-Functional Requirements

Table 3.3 Use Case Specifications

Table 4.1 Cost and Effort Estimation

Table 6.1 Input Validations and Checks

Table 7.1 Reports Generated by the System

Table 8.1 White Box Test Cases

Table 8.2 Black Box Test Cases

Table 9.1 Empirical Evaluation Summary

List of Figures

Figure 6.1 Login Screen

Figure 6.2 Dashboard After Analysis

Figure 7.1 Relationship Graph View

Figure 10.1 Gantt Chart

1. Title and Objective

1.1 Title

The title of the project is “Legal PDF Analyzer”. It is a web-based application designed to help users upload, process, and understand legal PDF judgments through automated reference extraction and relationship visualization.

1.2 Objective

The main objective of the project is to reduce the manual effort required to study long legal judgments and their references. A legal researcher often needs to know which earlier cases are referred to by a judgment, how different judgments are connected, and whether the extracted links are reliable. This project addresses those needs through a structured pipeline that converts PDF judgments into clean text, identifies reference patterns, matches them with available case documents, and presents the result in a dashboard, relation table, graph view, and validation screen.

The project also aims to provide a practical learning outcome for final-year computer science work: it combines web development, file handling, natural language processing, fuzzy matching, search indexing, authentication, reporting, and testing in one coherent application.

1.3 Scope

The scope of the system includes uploading legal PDFs, extracting text, cleaning extracted text, identifying citation-like references, matching parent-child case relationships, displaying analysis status, generating relation reports, viewing relationship graphs, validating confidence-based matches, and producing summaries or question-answer responses for selected documents. The system is intended for academic and research support, not as a replacement for professional legal advice.

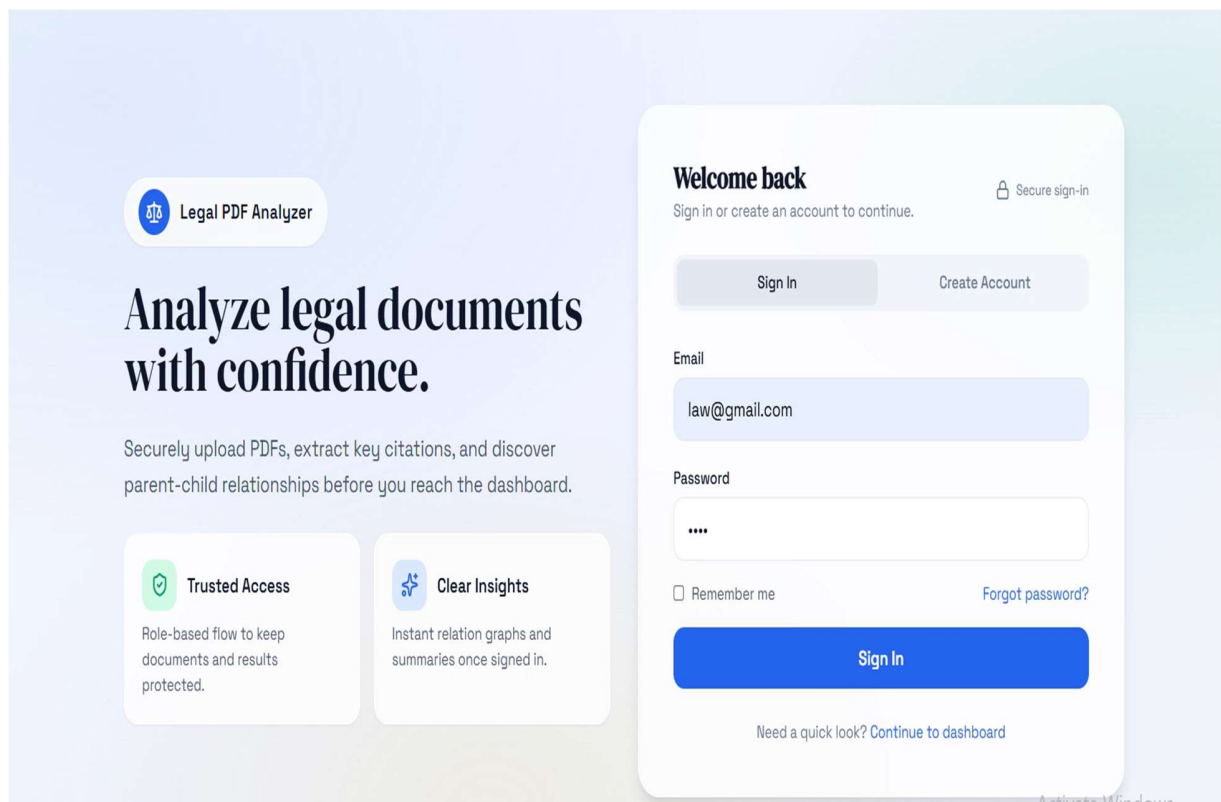
1.4 Brief Working of the Project

The project works like a guided research assistant for legal documents. The user first signs in and uploads PDF judgments. After the files are uploaded, the dashboard shows the document count and gives the user the option to start analysis. The system then processes the PDFs in the background and updates the status so the user does not have to manually execute separate steps.

Once analysis is completed, the application shows how many documents were processed, how many relationships were found, and what the average confidence score is. A relation in this project means that one document appears to refer to another document in the uploaded collection. The confidence score helps the user understand whether a link is likely to be strong or whether it should be reviewed carefully.

The output is not limited to a plain list. The user can open the relation table for detailed inspection, use the graph screen to understand document connectivity visually, and use the validation screen to review uncertain results. In this way, the application supports both quick overview and deeper manual checking.

Figure 1.1 Space for Project Home/Login Screenshot



2. Introduction and Problem Statement

2.1 Introduction

Legal judgments are usually lengthy, reference-heavy, and interconnected. A single judgment may refer to many earlier decisions, and the value of the document often depends on understanding those references. In traditional reading, a user must manually search for case names, compare references, open related documents, and build a mental map of relationships. This becomes slow and error-prone when the number of PDFs grows.

Legal PDF Analyzer was developed to make this workflow easier. The application treats legal PDFs as structured research material. It extracts text from judgments, cleans the extracted text, identifies citation-style references, matches references to available documents, and displays the resulting parent-child relationships in a usable interface.

2.2 Problem Statement

The problem addressed by this project is the difficulty of manually analyzing legal PDF judgments to identify important references and understand relationships between documents. Manual analysis consumes time, requires repeated searching, and does not provide a quick visual overview of how documents are connected. There is a need for a system that can accept legal PDFs, process them automatically, identify references, calculate match confidence, and present the output in a clear and verifiable format.

2.3 Existing System

In the existing manual approach, a user opens each PDF, searches for reference phrases, copies important case names into notes, and compares them with other documents. General PDF readers can search text, but they do not build case relationship tables or graphs. Search engines can locate a cited case externally, but they do not automatically connect the documents inside a local collection. This creates a gap for students, researchers, and legal professionals who want to analyze a curated set of judgments.

2.4 Proposed System

The proposed system provides a complete workflow from upload to insight. It allows authenticated users to upload legal PDFs, run the analysis pipeline, view extracted relations,

validate confidence levels, export relation data, and explore connected cases through a graph. The implementation uses Python and Flask for the backend, HTML/CSS/JavaScript templates for the interface, PyMuPDF for PDF text extraction, regular expressions and text normalization for reference detection, fuzzy/semantic matching for relation discovery, and Elasticsearch/MinIO support for indexing and scalable storage.

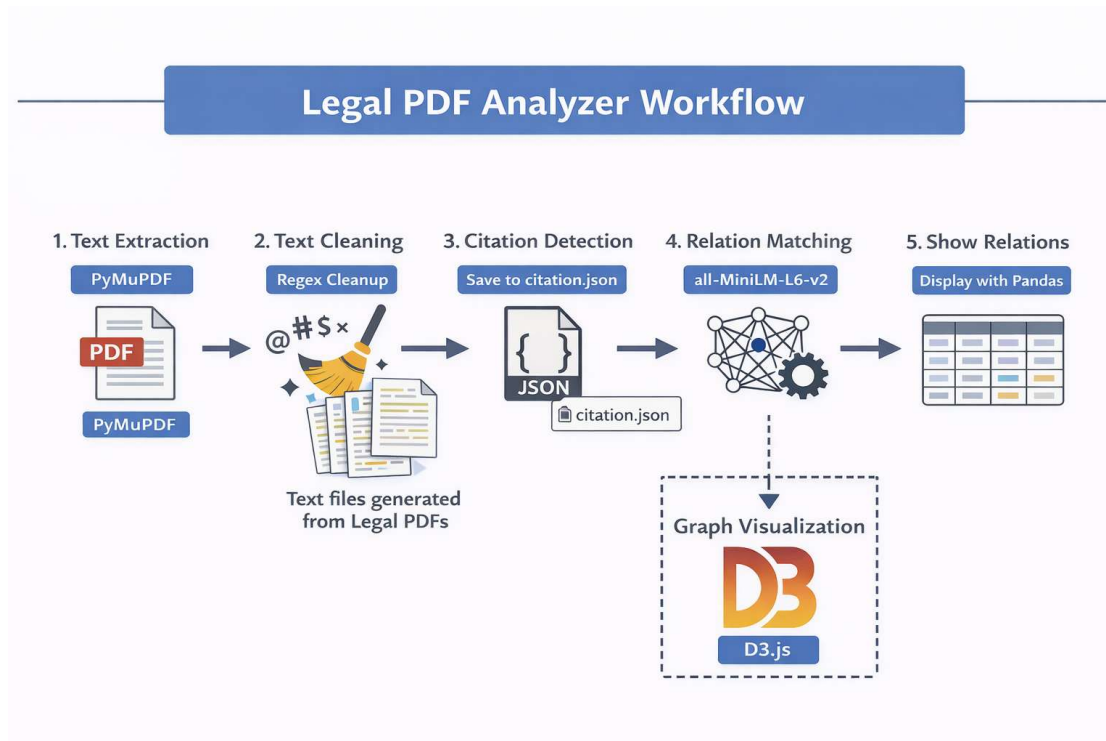
2.5 User Journey

From the user's point of view, the application is intentionally simple. The user is not expected to understand the internal pipeline. The screens guide the user from login to upload, from upload to analysis, and from analysis to reports. This makes the project suitable for students and researchers who want results without operating command-line tools.

Table 2.1 User Journey of the Proposed System

Stage	User Action	System Behaviour
Access	The user signs in or creates an account.	The system verifies the user and opens the dashboard.
Document Collection	The user uploads legal PDF files.	The system stores the files and shows them in the document list.
Processing	The user clicks Run Analysis.	The system extracts text, cleans it, identifies references, and builds relationships.
Review	The user opens reports or graph view.	The system presents relation counts, confidence values, and connected document views.
Validation	The user checks uncertain outputs.	The system supports manual review through confidence-based presentation.

Figure 2.1 Space for Dashboard Workflow Screenshot



3. Software Requirements Specification (IEEE Format)

3.1 Purpose

This SRS defines the functional and non-functional requirements of the Legal PDF Analyzer. It describes the expected behavior of the system, users, interfaces, constraints, and use cases.

3.2 Intended Users

The intended users include law students, legal researchers, advocates, academic reviewers, and computer science students studying document analysis or legal NLP. The administrator or project owner may also manage deployment, storage, and indexing services.

3.3 Product Perspective

The system is a web application with a Flask backend and a template-based frontend. It can work with local file storage and also includes optional MinIO object storage and Elasticsearch

indexing. The pipeline scripts are coordinated from the backend and generate structured outputs such as JSON and CSV files.

Table 3.1 Functional Requirements

Req. ID	Requirement
FR-01	The system shall allow a user to create an account and log in securely.
FR-02	The system shall allow PDF upload and duplicate checking.
FR-03	The system shall extract text from uploaded PDF judgments.
FR-04	The system shall clean extracted text and remove basic noise.
FR-05	The system shall detect legal reference patterns from cleaned text.
FR-06	The system shall match detected references with known case documents and calculate confidence.
FR-07	The system shall display dashboard statistics, document status, relation count, and average confidence.
FR-08	The system shall provide relation table, graph view, validation view, CSV export, summary, and document chat features.

Table 3.2 Non-Functional Requirements

Req. ID	Requirement
NFR-01	The interface shall be simple, readable, and suitable for repeated research use.
NFR-02	The system shall protect user sessions and hash stored passwords.
NFR-03	The pipeline shall provide progress/status feedback during analysis.
NFR-04	The relation output shall include confidence scores to support validation.
NFR-05	The architecture shall allow future storage/indexing extensions such as MinIO and Elasticsearch.

The required use case view is represented below in a compact tabular form. This keeps the report readable and avoids unnecessary technical drawings while still covering the actors, actions, and system responses expected in the guidelines.

Table 3.3 UML Use Case Summary

Actor	Use Case	System Response
User	Create account / login	Authenticates the user and opens the protected dashboard.
User	Upload legal PDF	Stores the document and shows it in the dashboard list.
User	Run analysis	Executes extraction, cleaning, reference detection, and relation matching.
User	View reports	Displays dashboard cards, relation table, graph, and validation view.
User	Export or review result	Allows CSV export and document-level inspection.

Table 3.4 Use Case Specifications

Use Case	Pre-condition	Basic Path	Alternate/Exception Path	Post-condition
Login	User has account	Enter email and password; submit login form	Invalid credentials show an error	User reaches dashboard
Upload PDF	User is logged in	Select one or more PDFs and upload	Duplicate/non-PDF file is rejected	PDF appears in document list
Run Analysis	At least one PDF exists	User starts pipeline; system extracts, cleans, detects, and matches	A failed step is shown in pipeline status	Relation and reference outputs are generated
View Graph	Relations are available	Open graph view and inspect connected nodes	If no relation exists, empty state is shown	User understands case network
Validate Result	Confidence scores are available	Open validation page and review confidence buckets	Low confidence items can be examined manually	User can trust or recheck results

3.4 Hardware and Software Requirements

The system can run on a normal development machine. Since PDF processing and semantic matching may consume memory, a machine with sufficient RAM is recommended for smooth execution. The exact requirement depends on the number and size of uploaded documents.

Table 3.5 Hardware and Software Requirements

Category	Requirement
Processor	Intel i5 or equivalent processor is recommended for local development.
Memory	Minimum 8 GB RAM; 16 GB RAM is better for larger PDF collections.
Storage	At least 2 GB free space for project files, PDFs, extracted text, and generated reports.
Operating System	Windows, Linux, or macOS with Python support.

Browser	Any modern browser such as Chrome, Edge, or Firefox.
Runtime	Python environment with required packages installed from requirements file.

3.5 External Interface Requirements

The user interface requirement is that the system should provide clear screens for authentication, document upload, analysis status, relation review, validation, and graph visualization. The file interface requirement is that the system should accept PDF documents and generate structured outputs such as cleaned text, relation data, and exportable CSV files.

The application also includes optional external service interfaces. Elasticsearch can be used for indexing and faster search-style retrieval. MinIO can be used as object storage for PDFs. These services are optional for extension and deployment, while the core academic demonstration can be understood through the local workflow.

3.6 Assumptions and Constraints

The system assumes that most uploaded PDFs contain extractable text. If a judgment is scanned as an image and does not contain text, the current implementation may not produce complete output without OCR. The system also assumes that legal references follow recognizable textual patterns. Very unusual formatting, spelling differences, or incomplete case names may reduce matching confidence.

A practical constraint is that automatic legal document analysis cannot be treated as final legal interpretation. The project helps organize and explore documents, but the final decision about the correctness of relationships should remain with the user. This is why confidence scores and validation screens are important parts of the design.

4. Cost and Effort Estimation

The project was developed as an academic final-year software project. The major cost is student effort, system configuration, and development time. Most software components are open-source, which keeps the financial cost low while still allowing a professional implementation.

Table 4.1 Cost and Effort Estimation

Activity	Estimated Effort	Remarks
Requirement analysis	1-2 weeks	Understanding legal PDF workflow and dissertation requirements
Backend pipeline	3 weeks	Extraction, cleaning, reference detection, relation matching
Frontend interface	2-3 weeks	Dashboard, upload, validation, relation table, graph views
Search/storage integration	1-2 weeks	Elasticsearch and optional MinIO support
Testing and debugging	2 weeks	Pipeline, upload, report, and UI verification
Documentation	1-2 weeks	Report preparation, diagrams, screenshots, bibliography

5. System Design and Diagrams

5.1 System Architecture

The architecture follows a simple layered structure. The user works through the browser interface. The backend manages login, upload, analysis, and report generation. The processing pipeline converts uploaded PDFs into clean text, identifies references, matches related documents, and returns the results to the dashboard, table, graph, and validation screens.

Table 5.1 System Architecture Summary

Layer	Main Responsibility	Output
User Interface	Login, upload, dashboard, validation, graph, relation view	Readable screens and user actions
Application Backend	Routes, authentication, file handling, status tracking	Secure workflow and API responses
Analysis Pipeline	Text extraction, cleaning, reference detection, relation matching	Structured relation output

Storage and Reports	PDF files, cleaned text, JSON/CSV results, optional search index	Documents, reports, and exportable data
---------------------	--	---

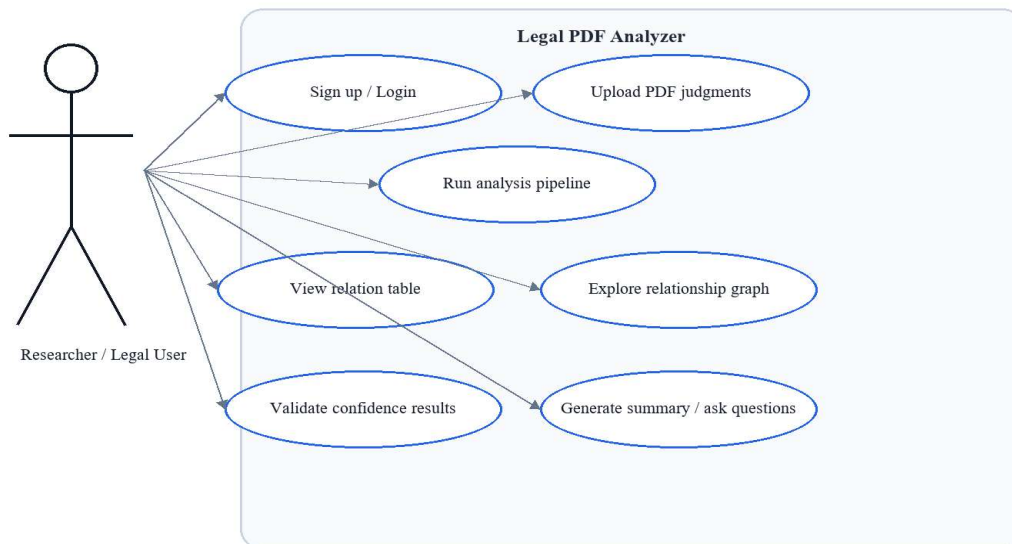
5.2 Class Diagram

The class design can be understood through the following project entities. These are described in a readable form rather than as a large technical picture.

Table 5.2 UML Class Summary

Class/Entity	Important Attributes	Important Operations
User	name, email, password hash, profile image	signup, login, logout, update profile
PDF Document	file name, file size, upload date, extraction status	upload, delete, preview, analyze
Reference Record	source document, reference text	detect, clean, remove duplicates
Case Relation	parent document, child document, confidence score	match, validate, export, visualize
Pipeline Run	steps, progress, status	start, track, reset, complete

UML Use Case Diagram

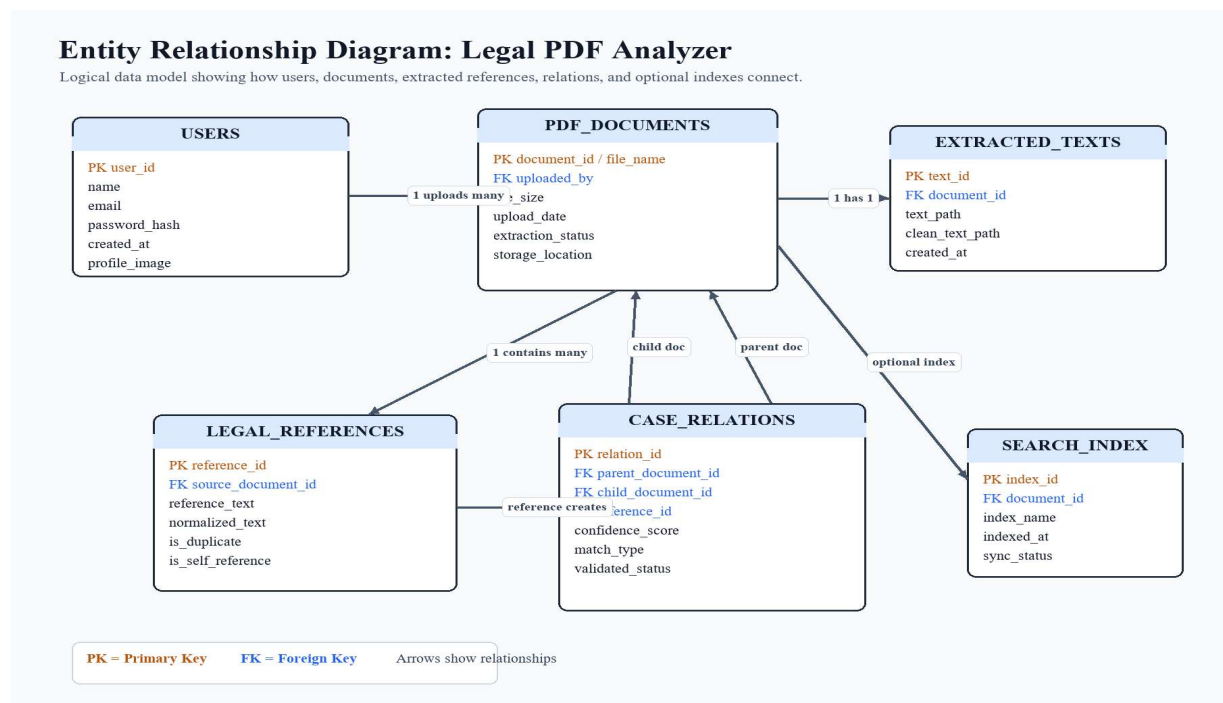


5.3 Entity Relationship Diagram

The ER view explains how the main records are connected. User records support authentication, PDF metadata represents uploaded documents, reference records store extracted references, and relation records connect one document with another through a confidence-based match.

Table 5.3 Entity Relationship Summary

Entity	Primary Key / Identifier	Relationship
users	id	One user may upload and analyze many documents.
pdf_metadata	file_name	One document may contain many detected references.
legal_references	reference_id	A detected reference may contribute to a relation.
case_relations	relation_id	A relation connects a parent document and a child document.



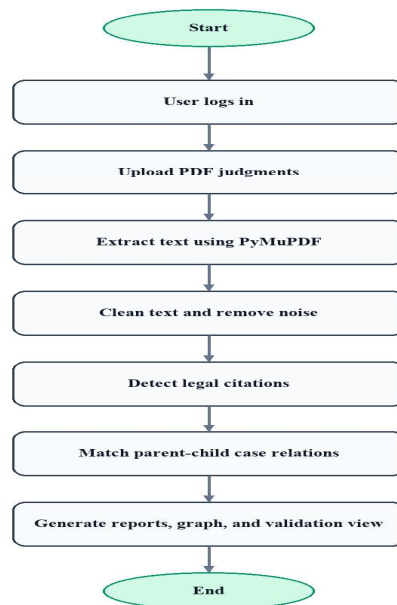
5.4 Activity Diagram

The activity flow is: login, upload PDF, run analysis, extract text, clean text, identify references, match relations, and review the generated dashboard, table, validation page, or graph. This flow keeps the user experience direct and avoids making the user run each processing step manually.

Table 5.4 Activity Flow Summary

Step	Description
Start	The user opens the application and reaches the login or sign-up screen.
Authentication	The user enters credentials and the session is created after successful verification.
Upload	The user uploads one or more PDF judgments into the system.
Pre-processing	The system extracts text and removes unnecessary spacing or page noise.
Reference Analysis	The system searches the cleaned text for legal reference patterns.
Relation Matching	The system compares references with available document titles and creates links.
Reporting	The dashboard, relation table, graph, and validation screens show the output.

Activity Diagram: Analysis Workflow



5.5 Module Description

The project is divided into modules so that every part of the workflow has a clear responsibility. This division also makes the system easier to test and improve. For example, the upload module can be tested separately from the relation matching module, and the graph view can be improved without changing the text extraction logic.

Table 5.5 Module-Wise Description

Module	Purpose	Main Output
Authentication Module	Handles sign-up, login, session control, and user identity.	Authenticated user session
Upload Module	Receives PDF files and keeps the document list updated.	Stored PDF documents
Extraction Module	Reads text from PDF pages and creates text files.	Extracted text
Cleaning Module	Normalizes spacing and removes basic noise from extracted content.	Clean text
Reference Detection Module	Finds legal reference-style phrases in the cleaned text.	Reference list
Relation Matching Module	Matches references to available documents and assigns confidence.	Parent-child relation table
Reporting Module	Displays summary cards, relation table, validation screen, and graph.	User-readable output

5.6 Data Flow Explanation

The data flow begins when a PDF is uploaded. The uploaded document becomes the input for the extraction process. The extracted text is then cleaned so that the next stage receives a more uniform text body. After cleaning, the system identifies references and compares them with available documents. The relation table is generated from these matches and becomes the base for dashboard statistics, graph visualization, validation, and export.

This flow is important because it separates raw data from processed data. A PDF is useful for reading, but the system needs structured outputs to build reports. The cleaned text, reference list, and relation table act as intermediate results that connect the original documents with the final user interface.

Table 5.6 Data Flow Summary

Input	Processing	Output
Uploaded PDF	Read text from pages	Extracted text file
Extracted text	Clean spacing and remove noise	Cleaned text file
Cleaned text	Identify reference patterns	Reference output
Reference output	Compare with known documents	Relation table with confidence
Relation table	Prepare UI and reports	Dashboard, graph, validation, export

6. Input Validations and Checks

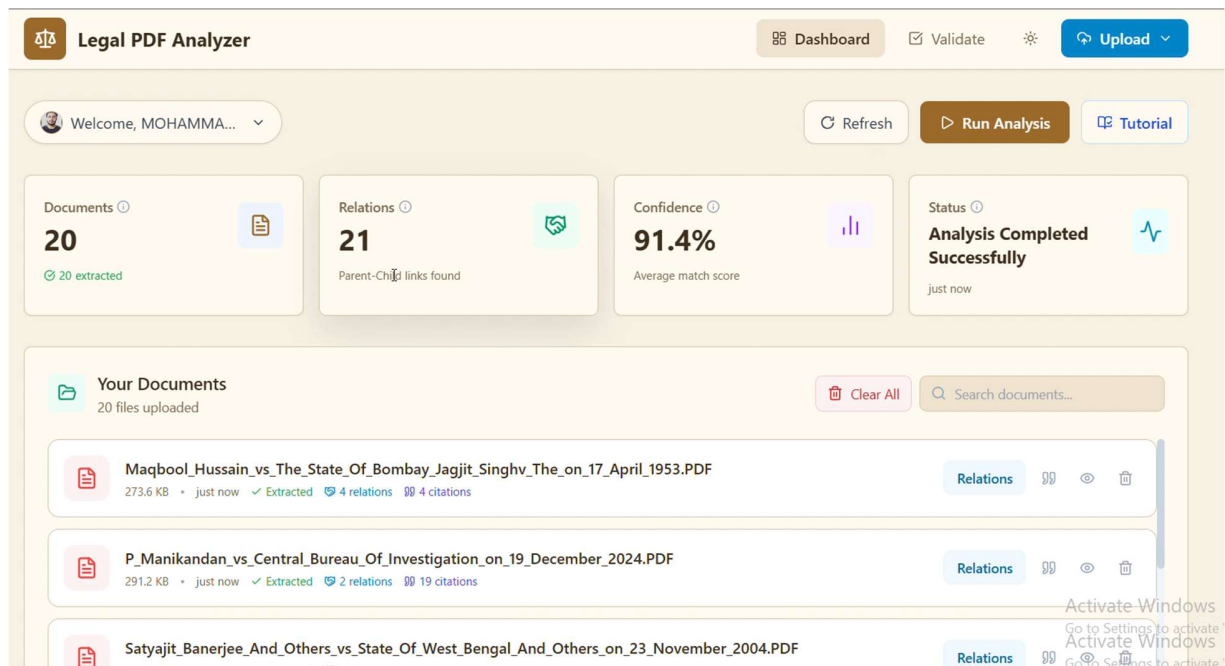
Input validation is important because the system deals with large user-supplied files and long-running processing steps. The implemented checks aim to protect the application from invalid files, duplicate uploads, missing data, failed extraction, stale outputs, and inconsistent search indexes.

Table 6.1 Input Validations and Checks

Input/Area	Validation or Check	Purpose
Authentication	Email uniqueness and password hashing	Prevents duplicate accounts and protects credentials
PDF Upload	File extension and secure filename handling	Allows valid PDF files and prevents unsafe names
Duplicate Files	Duplicate checking before upload	Avoids repeated processing of same documents
Pipeline	Step-wise status and error capture	Shows where processing fails

Extraction	No text found condition is handled	Avoids generating empty analysis output
Relations	Confidence score and deduplication	Supports validation and reduces repeated links
Deletion	Related text, reference, and relation outputs are pruned	Prevents stale reports after deleting files

Figure 6.2 Dashboard Summary After Analysis



7. Reports and Output Screens

The system generates both data reports and visual reports. The relation table is useful for precise review and export, while the graph view is useful for understanding case connectivity. The dashboard gives quick status information such as number of documents, relation count, average confidence, and analysis state.

Table 7.1 Reports Generated by the System

Report/Screen	Description	Use
Dashboard	Shows document count, extracted status, relation count, confidence, and pipeline controls	Quick project overview
Relations Table	Lists parent document, child document, reference example, confidence, and match type	Detailed review and CSV export
Graph View	Visualizes cases as nodes and references as edges	Network-style understanding of legal precedents
Validation View	Groups relation results by confidence level	Manual checking of uncertain matches
Document Preview	Displays PDF/text context, reference details, and relation information	Case-level investigation
Summary/Chat	Generates document summaries or answers questions from extracted content	Faster reading and understanding

Figure 7.1 Relationship Graph View

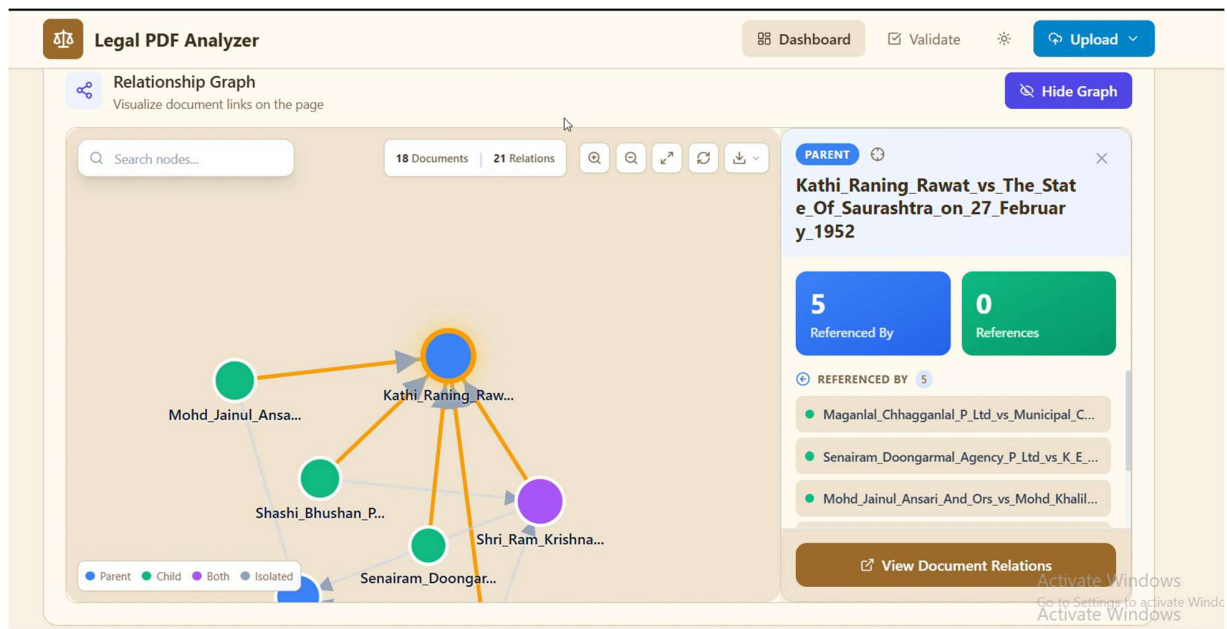
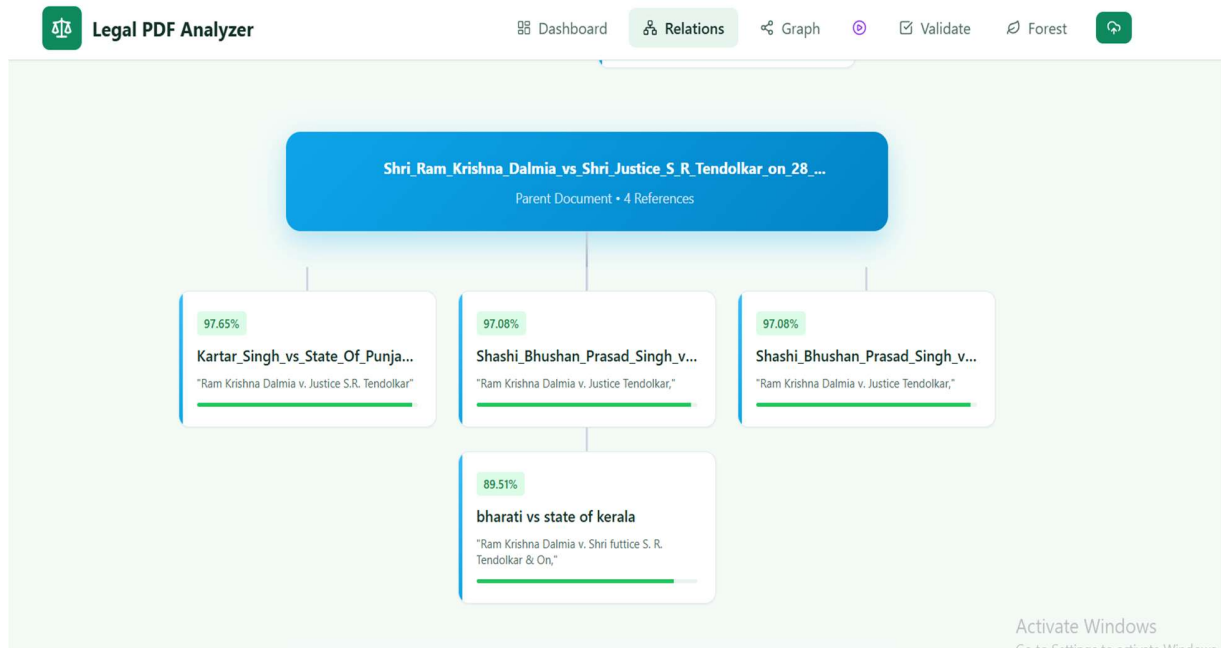


Figure 7.2 Space for Relation Table Screenshot



7.1 Explanation of Main Screens

The login screen gives the project a controlled entry point. It prevents casual access to the dashboard and makes the system feel like a complete application rather than only a script. The design is simple, with clear sign-in and account creation options.

The dashboard is the central working screen. It shows how many documents have been uploaded, how many have been processed, how many relationships were detected, and what the average confidence score is. This helps the user understand the project status at a glance.

The relation table is useful when the user wants detail. It presents the parent document, child document, reference example, confidence value, and match type. This table-style output is important because academic and legal users often need exact records rather than only visual summaries.

The graph view gives a visual understanding of document connectivity. Each node represents a document and the connection between nodes represents a detected relationship. This screen is especially useful when many documents are uploaded and the user wants to quickly see which documents are central in the collection.

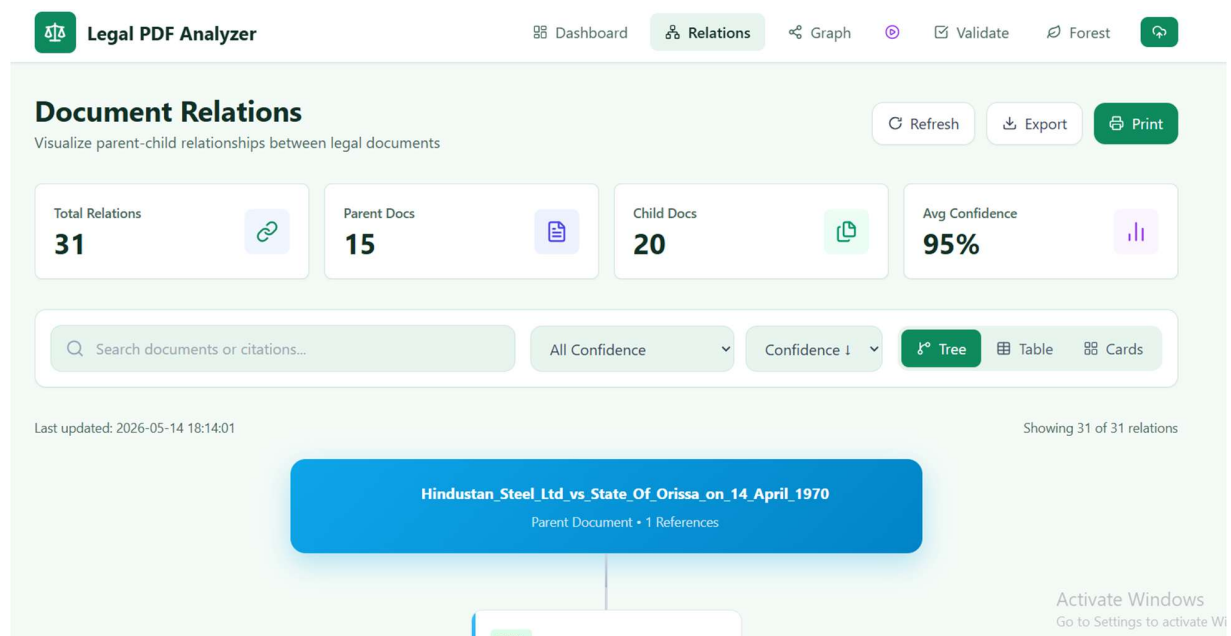
The validation screen supports manual checking. Since automatic analysis may not always be perfect, confidence-based review allows the user to focus first on uncertain outputs. This makes the system more responsible and practical.

7.2 Example Output Interpretation

For example, if Document A refers to Document B, the system can show Document B as a parent and Document A as a child. A high confidence value means the text match is strong and the relationship is likely to be correct. A lower confidence value does not mean the result is useless; it simply tells the user that the match should be reviewed before it is accepted.

This example-based interpretation keeps the project understandable without filling the report with long real case names. The main focus of the dissertation is the working of the software, not the legal content of any specific judgment.

Figure 7.4 Space for Example Output Screenshot



8. Testing

8.1 White Box Testing

White box testing was planned around internal logic where the implementation has clear paths: reference extraction and relation matching. For the reference detection module, independent

paths include valid reference match, duplicate removal, self-reference removal, and no reference found. For relation matching, independent paths include exact/fuzzy match, multi-criteria match, no valid match, and unmatched reference recording.

Table 8.1 White Box Test Cases

Module	Path/Condition	Test Data	Expected Result	Status
Reference Detection	Valid reference pattern exists	Text containing one case reference, for example 'A v. B'	Reference is extracted	Pass
Reference Detection	Duplicate reference appears twice	Same reference repeated in cleaned text	Only one reference is kept	Pass
Reference Detection	Reference refers to same filename	Self-reference in own judgment	Self-reference is removed	Pass
Relation Matching	High fuzzy and semantic match	Reference text similar to known case title	Relation is created with confidence	Pass
Relation Matching	No parent satisfies criteria	Unknown reference text	Reference goes to unmatched list	Pass

8.2 Black Box Testing

Black box testing focuses on user-visible behavior. Boundary Value Analysis and equivalence classes were considered for upload size, file type, login data, and pipeline state.

Table 8.2 Black Box Test Cases

Use Case	Technique	Input	Expected Output	Status
Login	Equivalence class	Registered email and correct password	Dashboard opens	Pass
Login	Equivalence class	Wrong password	Login error shown	Pass
Upload PDF	Boundary/extension check	Valid .pdf file	File is accepted	Pass

Upload PDF	Equivalence class	Non-PDF file	File is rejected or ignored	Pass
Run Analysis	Decision-based	No uploaded PDFs	No analysis output generated	Pass
Export Relations	Decision-based	Relation CSV exists	CSV download is available	Pass

8.3 Test Case Discussion

The testing approach is designed around the most important risks of the project. The first risk is invalid input, because the system accepts uploaded files. The second risk is incomplete processing, because the analysis pipeline has multiple steps. The third risk is incorrect interpretation, because relationship matching is based on text similarity and confidence scoring.

For this reason, testing is not limited to checking whether a page opens. The tests also check whether invalid uploads are handled, whether duplicate files are avoided, whether the pipeline reports status, whether output files are generated, and whether the user can inspect the results through multiple screens.

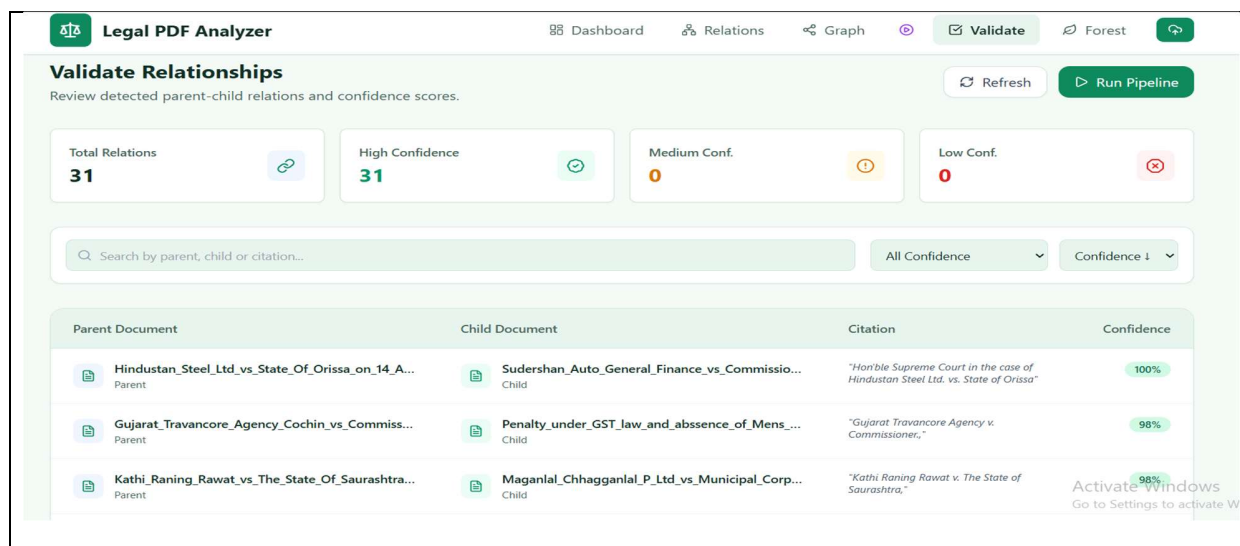
The white box tests focus on the internal decision paths. For instance, reference detection must behave differently when a valid reference exists, when a duplicate is found, when a self-reference appears, or when no reference is present. The black box tests focus on what the user experiences, such as logging in, uploading a PDF, starting analysis, and exporting results.

Table 8.3 Testing Risk Coverage

Risk	Testing Focus	Reason
Invalid file input	Upload validation	Prevents wrong file types from entering the analysis pipeline.
Empty or unreadable PDF	Extraction result check	Avoids misleading reports when no text is available.
Repeated document	Duplicate handling	Prevents unnecessary repeated analysis.
Uncertain relationship	Confidence review	Allows the user to verify doubtful outputs manually.
Long-running analysis	Pipeline status	Keeps the user informed during

		processing.
--	--	-------------

Figure 8.1 Space for Testing / Validation Screenshot



9. Empirical Evaluation

The repository contains a prepared sample dataset and generated analysis output. The observed output shows that the system can process a collection of legal judgments and produce measurable relation results. The confidence score helps separate strong matches from cases that may need manual review.

Table 9.1 Empirical Evaluation Summary

Metric	Observed Value	Interpretation
PDF documents available in repository sample	65	Input corpus for analysis
Clean text files available	65	Documents with extracted/cleaned text
Reference entries in JSON	97	Documents represented in reference output
Total references detected	2726	Raw reference discovery volume

Relations in relation_table.csv	59	Matched parent-child links
Average confidence	88.81%	Overall relation confidence level
Confidence range	77.5% to 98.33%	Shows variability for validation

In the screen recording used for this report, a demonstration run showed 20 uploaded documents, 21 detected parent-child relationships, and an average confidence of 91.4 percent. This indicates that the project is able to move from document upload to usable relation output in a complete workflow.

9.1 Evaluation Method

The empirical evaluation was performed by observing the output produced after running the project workflow on a collection of legal PDFs. The evaluation does not attempt to make a legal judgment about each document. Instead, it measures whether the software workflow is complete, whether outputs are generated, and whether those outputs are understandable to the user.

The main indicators used in evaluation are document count, processed text count, number of detected references, number of generated relations, and confidence range. These indicators are suitable for a software project because they show both processing coverage and output usefulness.

9.2 Evaluation Observations

The dashboard output is useful because it gives an immediate summary after analysis. The relation count helps the user know whether the uploaded collection has meaningful connections. The confidence score gives a quick idea of how strong the matches are. The graph view then turns those numerical outputs into a visual network.

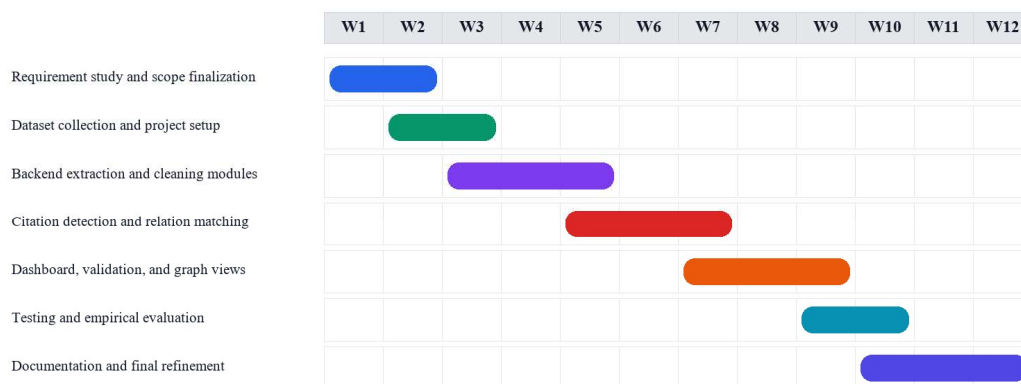
During observation, the project performed best when the PDF text was clean and the referenced case names were close to the document names available in the collection. This is expected because the system relies on text extraction and similarity-based matching. The validation screen is therefore an important part of the project because it accepts that automatic matching should be reviewable.

10. Gantt Chart

The following Gantt chart depicts the project timeline. The work moved from requirement analysis to implementation, testing, and documentation in phases.

Figure 10.1 Gantt Chart Depicting Timeline of Activities

Gantt Chart of Project Activities



The timeline was planned so that the core workflow could be completed first. Requirement analysis and dataset collection were done before the main implementation. After that, the backend pipeline and frontend screens were developed together so that the user interface could reflect the actual processing status. Testing and documentation were kept toward the end, but corrections were made throughout the project whenever an issue was found.

10.1 Project Management Notes

A final-year project requires both technical progress and documentation discipline. For this project, the work was divided into small milestones: understanding the problem, preparing the repository, building the upload and authentication flow, implementing the analysis pipeline, designing the reports, testing the workflow, and preparing the dissertation.

This staged approach reduced confusion because every milestone had a visible output. For example, after the upload module was completed, the dashboard could show uploaded documents. After the analysis pipeline was connected, the dashboard could show processing results. After graph integration, the relation output became easier to understand visually.

Table 10.1 Project Milestones

Milestone	Deliverable
Requirement finalization	Problem statement, objective, scope, and expected users
Core backend	PDF extraction, text cleaning, reference detection, relation matching
Frontend screens	Login, dashboard, relation view, graph view, validation view
Testing	White box and black box test cases with expected results
Documentation	Dissertation report, tables, figures, and final formatting

11. Conclusion

The Legal PDF Analyzer project successfully demonstrates how software engineering and natural language processing techniques can be applied to a practical legal research problem. The system accepts legal PDF judgments, extracts and cleans text, detects important references, matches parent-child relationships, and presents the output through a dashboard, relation table, graph, and validation interface.

The most important achievement of the project is that it converts a slow manual reading task into an organized digital workflow. Instead of only opening PDFs one by one, the user can see references, confidence scores, connected cases, and document-level insights in one place. The project is also extensible because it includes support for Elasticsearch indexing, optional MinIO storage, Docker deployment, and summary/chat features.

From a learning perspective, the project helped connect several areas of computer science: web application development, file processing, data cleaning, pattern recognition, similarity matching, database-backed authentication, visualization, and software testing. It also showed that a useful project does not need to solve every legal problem completely; it can still be valuable by reducing repetitive work and presenting information in a clearer way.

12. Limitations

The system depends on extractable PDF text. Scanned documents without OCR may not produce usable text. Reference detection is based on legal text patterns and may miss unusual citation formats. Relation matching can produce uncertain results when case names are abbreviated, incomplete, or very similar. The semantic model may take time to load during the first run. The current project is suitable for academic and research support, but legal conclusions should still be verified by a qualified person.

Table 12.1 Limitations and Handling

Limitation	Impact	Possible Handling
Scanned PDFs	Text may not be extracted correctly.	Add OCR support in future versions.
Unusual reference format	Some relationships may be missed.	Improve parsing rules and training data.
Similar case names	Confidence may be uncertain.	Use validation and manual review.
Large document collection	Processing may take more time.	Use indexing and background jobs.
Legal interpretation	System output cannot replace expert judgment.	Treat output as research assistance.

13. Future Scope

Future work can add OCR support for scanned judgments, stronger legal reference parsers, better model evaluation against a manually verified benchmark, role-based administration, advanced graph filtering, multilingual judgment handling, automatic bibliography generation, deployment hardening, and a more complete test suite. The system can also be extended with court-wise metadata extraction, timeline analysis, and precedent strength ranking.

Another useful improvement would be a more advanced validation workflow where the user can mark a relation as correct, incorrect, or uncertain. These manual decisions could later be used to

improve the matching logic. A full admin panel can also be added for managing users, uploaded documents, and project-level settings.

Table 13.1 Future Scope Summary

Future Feature	Benefit
OCR for scanned PDFs	Allows analysis of image-based judgments.
Advanced validation labels	Improves trust and supports future model improvement.
Role-based access	Makes the system suitable for teams or institutions.
Better graph filters	Helps users explore large document networks.
Court/date metadata extraction	Adds richer legal research context.
Benchmark dataset	Allows more formal accuracy evaluation.

14. User Manual

This section gives a short operating guide for the Legal PDF Analyzer. It is included so that a new user can understand the expected flow of the project without reading the internal implementation.

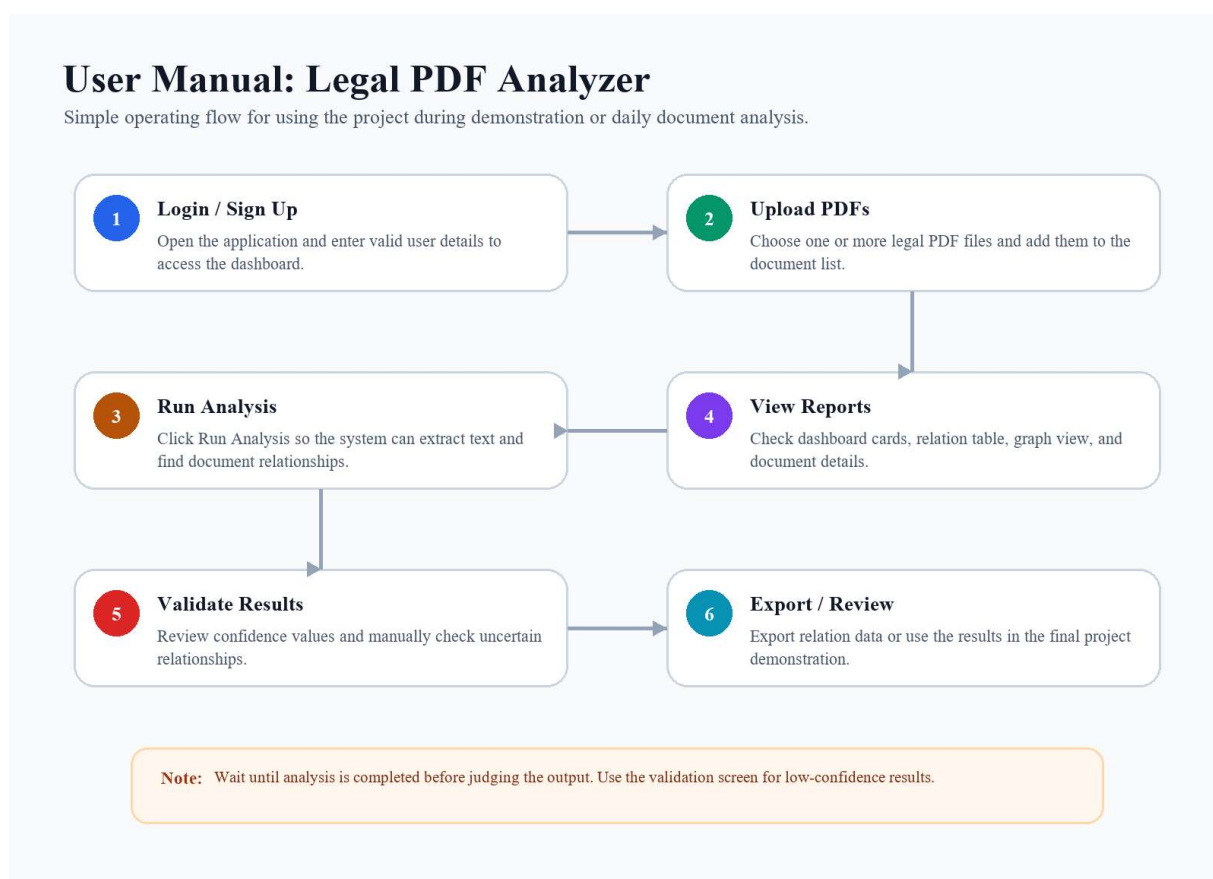
Table 14.1 User Manual Steps

Step	Action	Expected Result
1	Open the application in the browser.	The login/sign-up screen is displayed.
2	Sign in with valid credentials.	The dashboard opens with upload and analysis options.
3	Upload one or more legal PDF files.	The document list is updated on the dashboard.
4	Click Run Analysis.	The system starts processing and updates the status.
5	Open relation, graph, or validation screens.	The processed output is shown in different formats.
6	Review confidence values and export	The user can preserve or verify the analysis output.

	results if required.	
--	----------------------	--

The most important user instruction is to wait until the analysis status shows completion before judging the output. If the output appears empty, the user should first check whether the uploaded PDFs contain selectable text and whether the analysis process completed successfully.

Figure 14.1 Space for Step-by-Step User Manual Screenshot



For viva or demonstration, the suggested flow is: show the login page, upload or display sample documents, run the analysis, explain the dashboard cards, open the relation table, and finally show the graph view. This sequence explains the project clearly without going into unnecessary implementation detail.

15. Bibliography

- [1] IEEE Computer Society, IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998.
- [2] G. van Rossum and Python Software Foundation, Python Language Reference, Python Software Foundation, 2026.
- [3] Pallets Projects, Flask Web Framework Documentation, 2026.
- [4] Artifex Software, PyMuPDF Documentation, 2026.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proc. EMNLP-IJCNLP, 2019.
- [6] Elastic, Elasticsearch Guide, Elastic Documentation, 2026.
- [7] Docker Inc., Docker Documentation, 2026.
- [8] MinIO, MinIO Object Storage Documentation, 2026.
- [9] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed., Addison-Wesley, 2003.
- [10] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach, 9th ed., McGraw-Hill, 2019.